# 后续工作交接（2026-10-06 收工）

> **给明天开工的自己。**这份文档只写「现在在哪、下一步做什么、怎么验证」， 不重复历史复盘。 ⚠ **最重要的一条**：训练语料**还没按 v3 的实体重做**， 而 v3 需要 52 个机场三字码、训练语料只覆盖 20 个 —— **33 个是模型从未见过的。**不补的话 v3 上的 `flight_search` 会重演 「编造三字码」。这条必须在跑 SFT 之前做完（见 §2 的 B0 步骤）。

---

## §1 当前状态

### 1.1 已可信的部分

**任务集 v3（106 个任务，约束可检查）**

| 判据 | v1（已废弃） | v3（当前） |
|---|---|---|
| 骨架/任务比 | 0.52 | **0.99** |
| `multi_tool` 骨架比 | 0.11 | **1.00** |
| `multi_tool` 要求工具数 | 1 | **2** |
| 订单号池 | 8 个（复用 7 次） | **32 个全唯一** |
| 乘客名池 | 6 个 | **18 个全唯一** |
| 日期区间 | 2025-07~08（**过去**） | **2026-11~2027-01（未来）** |
| seen/unseen 骨架交集 | 6（26%） | **0** |
| 跨类别实体重合 | 未检查 | **0** |
| 空参数任务（永远判成功） | 2 个 | **0** |
| clarify 样本 | 4 | **14** |
| 真多轮任务 | 0 | **6** |
| 真纠错任务 | 0 | **15** |

类别构成：

```
cancellation      22    (8 带原因 + 14 应先追问)
booking_flow      15
error_correction  15    (原 recovery，改名为「纠错」)
baggage           14
flight_search     14
multi_tool        10
weather_advisory  10
multi_turn         6    (真多轮：用户改主意 → 必须跨轮更新参数)
```

**质量保障**

`103` 个单元测试 · 全模块 `doctest` · `mock` 全矩阵

`7` 条结构性约束 + 「约束检查器能真的报警」的反向测试

**污染防线（四层，全部在代码里）**

① `EVAL_TASK_PREFIX = "trainsyn_"` 定义训练/评测分界线

② `build_all` 硬门禁：越界即抛 `ValueError`（报错里写明「会让全部结论作废」）

③ 默认 `tasks=[]`：忘记传参也不会污染

④ 调用侧只传合成语料 + 打印数据源

⑤ `test_training_data_never_contains_eval_tasks`（5 条断言）

## 1.2 全部作废的数字

**这些数字不可再引用**（要么跑在 v1 任务集上，要么用了污染数据）：

| 数字 | 作废原因 |
|---|---|
| baseline 50.25% | 跑在 v1 上 |
| sft_only 95.00% | **污染数据** |
| sft_v2 78.75% / sft_v3 83.00% | 跑在 v1 上 |
| dpo_joint 68.5% / dpo_from_sft 10.0% | 污染数据 |
| dpo_sft_v3_b05 49.25% | 干净数据，但跑在 v1 上 |
| agent 评测 100% / 89.74% | 跑在 v1 上 |

## 1.3 项目要说明什么（一句话）

**当偏好数据编码的是「全局风格倾向」而不是「条件性规则」时， DPO 会让模型过度泛化它，从而损害任务执行能力。** 而且：**纯正例监督（SFT）无法编码否定性约束** —— 证据是 `missing_reason`（信息不足应先追问）上 SFT **0/4**、DPO **4/4**。 但这个证据目前 **n=4**，v3 已把它提到 **n=14**。

**三条待证伪的命题**（v3 上的实验就是为它们设计的）：

| # | 命题 | 如何证伪 |
|---|---|---|
| 1 | 损害是 **clarify 偏好对**造成的，不是 DPO 本身 | 剂量-反应：clarify 占比 0% / 30% / 100% |
| 2 | 用**条件化**偏好数据能修好 | 同一动作在两个上下文里结论相反的数据 |
| 3 | 改 prompt 拿不到同样效果（否则 DPO 多余） | 不改权重，只在 prompt 里加规则 |

---

# §2 明天的工作顺序

## 阶段 A：拿到 v3 上的可信基线（必须先做）

| 序 | 任务 | 谁 | 时间 | 前置 |
|---|---|---|---|---|
| **A1** | 更新代码到云端 | 你 | 5 分钟 | — |
| **A2** | 跑 v3 baseline | 你 GPU | 15 分钟 | A1 |
| **A3** | 按实测准确率重标难度 | 我 | 30 分钟 | A2 |

**A1 命令**

```
cd /root/autodl-tmp/agent-optimization
python scripts/deploy/update_in_place.py \
    /root/autodl-tmp/agent-optimization-upload.zip --also-preferences
python tests/run_tests.py            # 预期 103 通过
```

**A2 命令**

```
python -u scripts/eval/run_baseline.py \
    --label baseline_v3 --task-set v3 --num-samples 4 \
    2>&1 | tee baseline_v3.log
```

⚠ **不要加 `| tail -N`** —— 那会隐藏全部输出直到结束。

**A3 我要写的脚本**：`scripts/eval/assign_difficulty_by_accuracy.py`

- 读 `experiments/baseline_v3/eval_report.json` 的逐任务准确率
- 按分位数重标 easy/medium/hard
- 输出「原标签 vs 实测标签」对照表（看差异有多大）
- **理由**：现在是按类别拍脑袋赋难度（baggage 全 easy、`flight_search` 全 hard），

这是专家明确指出且尚未修复的一条

## 阶段 B：让训练语料匹配 v3（B0 是阻塞项）

| 序 | 任务 | 谁 | 时间 | 为什么必须 |
|---|---|---|---|---|
| **B0** | **扩展合成语料的实体覆盖** | 我 | 60 分钟 | ⚠ **阻塞**：v3 要 52 个机场码，训练只有 20 个，**33 个没见过** |
| B1 | 重训 SFT（v3 上的干净模型） | 你 GPU | 25 分钟 | B0 |
| B2 | 重评 SFT + 出错误分布 | 你 GPU | 15 分钟 | B1 |

**B0 的具体内容**：把 `src/data/recovery_corpus.py` 的合成语料里的机场码池

从 20 个扩到 60 个（与 v3 的 `AIRPORT_CODES_V2` 对齐），并加一条断言：

```
# 断言：v3 评测用到的每个机场码，训练语料里都必须出现过
assert not (v3_codes - syn_codes), f"训练缺失这些机场码：{v3_codes - syn_codes}"
```

**为什么这条要写成断言**：这个坑已经踩过一次（`cancel_booking` 在合成语料里

完全没出现，被污染数据掩盖了）。**「训练没见过 → 必然编造」是确定性失败，**

不该靠人记得检查。

**B1 命令**（B0 完成后）

```
nohup python -u scripts/train/sft_train.py \
    --experiment sft_only --output-dir outputs/sft_v4 \
    > sft_v4.log 2>&1 &
```

**B1 训练时要确认的三项**（都在日志开头）

| **检查** | **期望** |
|---|---|
| 训练样本数 | 与 B0 后的语料条数一致 |
| **截断率** | **0.0%** ← 之前 23.2% 静默截断过 |
| 按类型分布 | `format_constraint` / `tool_selection` / `param_constraint` 都非零 |

## 阶段 C：机制实验（项目的核心）

**三个实验都设计成「无论结果如何都有产出」。**

| **序** | **实验** | **配置** | **时间** | **判定** |
|---|---|---|---|---|
| **C1** | **受控剂量-反应** | 同一 sft_v4 起点、同 β、同 lr，只改 clarify 对占比：**0% / 30% / 100%** | 45 分钟 | 若 `clarify_instead` 单调升、accuracy 单调降 → **机制确立** |
| **C2** | **条件化偏好数据** | 构造「同一动作在两个上下文里结论相反」的偏好对，训练 | 我 60 分钟 + 你 30 分钟 | 若过度澄清消失且 accuracy 恢复 → **修复成立** |
| **C3** | **prompt 对照** | 不改权重，只在 prompt 里加「信息不足才追问」的规则 | 你 20 分钟 | 若效果相同 → **DPO 是多余的** |

**三种结果组合都有明确产出**

| **C1** | **C2** | **C3** | **结论** |
|---|---|---|---|
| ✅ | ✅ | ❌ | **方法学贡献**：「数据形式决定成败」+ 可操作的构造规则 |
| ✅ | ❌ | — | **更强的负面结论**：1.5B 上条件化偏好不可学 |
| ❌ | — | — | 我的机制假设错了，需重新定位（**如实说**） |

**C1 的实现要点**（我写，约 90 分钟）

`scripts/train/dpo_dose_response.py`

- 三个变体：`clarify` 占比 `0%` / `30%` / `100%`
- 严格共用：同一起点（`outputs/sft_v4/final`）、同 β、同 `lr`、同 `epoch`
- 只改 `clarify` 偏好对的数量
- 产物：`outputs/dpo_dose_{0,30,100}/`

### 阶段 D：报告与统计工具（可并行，不需要 GPU）

| 序 | 任务 | 时间 | 说明 |
|---|---|---|---|
| D1 | **重构消融报告** | 40 分钟 | 主表只用 v3 数字；v1 数字下沉附录并标明不可引用；结论分 A/B/C 三档 |
| D2 | `paired_significance.py` | 40 分钟 | McNemar + **配对** bootstrap + 功效分析 + MDE |
| D3 | `measure_kl_drift.py`（**轨迹级**，不是 token 级） | 60 分钟 | 替代 `logps/chosen` 判据 |
| D4 | `llm_grader_check.py` | 60 分钟 | 规则 grader vs LLM judge 的一致率（Cohen's κ） |
| D5 | 更新简历条目与面试稿 | 40 分钟 | 当前版本还引用着作废数字 |

**D2 为什么重要**：我一直在用「CI 是否重叠」判断显著性 —— **那是错的**，

CI 重叠 ≠ 无差异。要用**同一批任务上的配对检验。**

**D3 为什么重要**：我原脚本是 token 级 KL，而 DPO 的 KL 约束在

**整个序列分布**上。而且 `logps/chosen` 已被证明会漏报

（β=0.5 时它稳定，accuracy 崩了 33.75pp）。

### 阶段 E：可选扩展（时间允许）

| 序 | 任务 | 时间 | 价值 |
|---|---|---|---|
| E1 | 7B 模型对照 | 3 小时 GPU | 回答「1.5B 的失效结论能否外推」 |
| E2 | ORPO / KTO 对照 | 1 小时 GPU | 验证崩塌是否源于 DPO 的 reference 机制 |
| E3 | 复现性重跑（同配置两次） | 30 分钟 GPU | 给出复现性误差范围 |
| E4 | agent 评测在 v3 上重跑 | 30 分钟 GPU | v3 有 `multi_turn` 类，agent 循环测它更合适 |

---

## §3 时间与成本估算

| 阶段 | 我的工时 | GPU 工时 | 成本（3090 ¥1.68/h） |
|---|---|---|---|
| A（基线） | 30 分钟 | 15 分钟 | ≈ ¥0.5 |
| B（语料 + SFT） | 60 分钟 | 40 分钟 | ≈ ¥1.2 |
| C（机制实验） | 90 分钟 | 95 分钟 | ≈ ¥2.7 |
| D（报告 + 工具） | 3.5 小时 | 0 | ¥0 |
| **合计（到 D）** | **约 6 小时** | **约 2.5 小时** | **≈ ¥4.5** |
| E（可选） | 1 小时 | 约 5 小时 | ≈ ¥8.5 |

---

## §4 命令速查

### 环境检查

```
cd /root/autodl-tmp/agent-optimization
python tests/run_tests.py                    # 103 通过
python -c "
import sys; sys.path.insert(0,'.')
from src.data.task_manifest import TASK_SET_SIZES, load_tasks_by_version
print(TASK_SET_SIZES)
for v in ('v1','v3'): print(v, len(load_tasks_by_version(v)))
"
# 预期：{'v1': 100, 'v3': 106}   v1 100   v3 106
```

### 验证任务集约束（不跑模型）

```
python -c "
import sys; sys.path.insert(0,'.')
from src.data.task_manifest_v2 import verify_task_set
from src.data.task_set_v2_specs import build_specs
r = verify_task_set(build_specs())
print('任务数', r['n'], '骨架比', r['skeleton_ratio'])
print('违规', r['violations'] or '无')
"
# 预期：任务数 106 骨架比 0.99 违规 无
```

### 验证训练语料实体覆盖（B0 之后必须为 0 缺失）

```
python -c "
import sys; sys.path.insert(0,'.')
from src.data.task_manifest_v2 import load_tasks_v3
from src.data.recovery_corpus import build_training_tasks
v3 = load_tasks_v3(); syn = build_training_tasks(seed=42)
syn_codes = {t.expected.args.get(k) for t in syn
             for k in ('origin','destination') if t.expected.args.get(k)}
v3_codes = {t.expected.args.get(k) for t in v3
            for k in ('origin','destination') if t.expected.args.get(k)}
miss = sorted(v3_codes - syn_codes)
print('训练缺失机场码:', len(miss), miss[:10])
"
# 当前：33 个缺失（B0 要修到 0）
```

### 跑 v3 baseline

```
python -u scripts/eval/run_baseline.py \
    --label baseline_v3 --task-set v3 --num-samples 4 \
    2>&1 | tee baseline_v3.log
```

### 训练监控（跑完自动报告）

```
LOG=sft_v4.log
for i in $(seq 1 80); do
  if ! pgrep -f sft_train.py >/dev/null; then
    echo "———————— 进程退出 ————————"
    if [ -d outputs/sft_v4/final ]; then
      echo "✓ 正常完成"; tail -6 "$LOG"
    else
      echo "✗ 异常退出："; tail -20 "$LOG"
    fi
    break
  fi
  echo "$(date +%H:%M:%S)  $(grep -oE '[0-9]+/[0-9]+ \[[0-9:]+<[0-9:]+' "$LOG" | tail -1)"
  sleep 30
done
```

⚠ **判断训练是否完成必须看 `outputs/.../final` 是否生成**，

不能只看进程 —— 进程消失可能是 OOM 崩溃而不是完成（这个坑踩过）。

---

## §5 三件我必须提醒自己的事

### 5.1 不要再相信「看起来对」

这个项目里 **8 个最严重的缺陷都不报错**：

| 缺陷 | 表面现象 |
|---|---|
| 训练数据污染 | 准确率**更高** |
| 空参数任务 | 那 2 个任务**永远成功** |
| 一致性检查假通过 | 报「✓ 一致」 |
| 循环导入测试不注册 | 测试数少 7 个但**不报错** |
| prompt 截断 23.2% | 报告「健康」 |
| TRL 指标键名错 | 显示 NaN（像「没记录」） |
| 内建 `hash()` 做种子 | 每次数据不同但条数相同 |
| YAML 类型分歧 | 本地正常、云端崩 |

**共同点**：不报错、数字正常、结论是错的。

**所以每加一个检查，都要问：「它能失败吗？」 并真的喂一条已知坏数据试一次。**

### 5.2 别再用「容易写的判据」代替「正确的判据」

我在这上面错了两轮：

| 我写的判据 | 为什么错 |
|---|---|
| 「天气城市不得与其他类别重合」 | 防的是**同一组城市对**复用，不是**同一个城市名。**城市组合空间上千组，重合是自然且不可避免的 |

| | |
|---|---|
| 「recovery 的 (工具,参数) 不得与其他类相同」 | tool=None, args={} 的空签名让所有「应追问」任务互相误报 |

**两次都是我把判据写得比问题更宽，然后被自己的检查器「违规」警告带偏。**

### 5.3 改动范围必须先确认

我因为「用 index() 切掉我以为的区间」把 load_tasks 一起删了，整个测试套件崩掉。

**这次是测试立刻报错才发现的，不是因为我有什么防范措施。**

**规则**：用 index() / replace() / 切片前，先打印要切的首尾各 3 行确认。

---

## §6 一句话

**明天先做 A（拿到 v3 基线）和 B0（补训练语料的实体覆盖）——**

这两件不做完，后面所有实验都是在错的地基上盖楼。